

第6章 计算几何

几何问题是高水平算法竞赛中不可或缺的题型。这类问题背景知识庞杂，内容难度较高。本章将介绍其中一些常见算法的实现代码，希望能给读者提供一些相对精简、规范的样例模板，以帮助读者在竞赛中快速解决几何相关问题。

6.1 二维几何基础

本节通过一些经典题目的实现代码，向读者展现点、直线、向量等二维几何常见基础问题的相关算法实现。

例 6-1 【线段相交；角度计算】 Morley 定理（Morley's Theorem, UVa 11178）

Morley定理是这样表述的：作 $\triangle ABC$ 每个内角的三等分线，相交成 $\triangle DEF$ ，则 $\triangle DEF$ 是等边三角形。

所有坐标均为不超过1000的非负整数。如图6-1所示，你的任务是根据 A 、 B 、 C 3个点的位置，确定 D 、 E 、 F 3个点的位置。

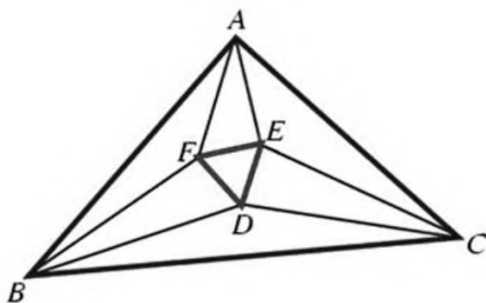


图6-1 Morley定理示意图

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

typedef Point Vector;
Vector operator+(const Vector& A, const Vector& B) {
    return Vector(A.x + B.x, A.y + B.y);
}
Vector operator-(const Point& A, const Point& B) {
    return Vector(A.x - B.x, A.y - B.y);
}
Vector operator*(const Vector& A, double p) { return Vector(A.x * p, A.y * p); }
double Dot(const Vector& A, const Vector& B) { return A.x * B.x + A.y * B.y; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
double Angle(const Vector& A, const Vector& B) {
    return acos(Dot(A, B) / Length(A) / Length(B));
}
double Cross(const Vector& A, const Vector& B) { return A.x * B.y - A.y * B.x; }
Point GetLineIntersection(const Point& P, const Point& v, const Point& Q, const
Point& w) {
    Vector u = P - Q;
    double t = Cross(w, u) / Cross(v, w);
    return P + v * t;
}
Vector Rotate(const Vector& A, double rad) {
    return Vector(A.x * cos(rad) - A.y * sin(rad), A.x * sin(rad) + A.y * cos(rad));
}
istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
ostream& operator<<(ostream& os, const Point& p) { return os << p.x << " " << p.y; }
Point getD(Point A, Point B, Point C) {
    Vector v1 = C - B;
    double a1 = Angle(A - B, v1);
    v1 = Rotate(v1, a1 / 3);

    Vector v2 = B - C;
    double a2 = Angle(A - C, v2);
    v2 = Rotate(v2, -a2 / 3);

    return GetLineIntersection(B, v1, C, v2);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    for (Point A, B, C; T--;) {
        cin >> A.x >> A.y >> B.x >> B.y >> C.x >> C.y;
        Point D = getD(A, B, C), E = getD(B, C, A), F = getD(C, A, B);
        printf("%.6lf %.6lf %.6lf %.6lf %.6lf %.6lf\n",
            D.x, D.y, E.x, E.y, F.x, F.y);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.1.4 节例题 1
注意：cin 和 scanf 不要混用，printf 和 cout 也不要混用，但是输入与输出是独立无关的
同类题目：Triangle Fun, UVa 11437
*/

```

例6-2 【线段相交；欧拉定理】好看的一笔画（That Nice Euler Circuit, 上海2004, POJ 2284）

平面上有一个包含 n ($4 \leq n \leq 300$) 个端点的一笔画，第 n 个端点总是和第一个端点重合，因此图案是一条闭合曲线。组成一笔画的线段可以相交，但是不会部分重叠，如图6-2所示。求这些线段将平面分成了多少个部分（包括封闭区域和无限大区域）。



图6-2 一笔画示例

【代码实现】

```

// 刘汝佳
#include <cmath>
#include <cstdio>
#include <iostream>
#include <vector>
#include <set>
using namespace std;
#define _for(i,a,b) for( int i=(a); i<(int)(b); ++i)

typedef long long LL;
const double eps = 1e-10;
int dcmp(double x) { if (fabs(x) < eps) return 0; return x < 0 ? -1 : 1; }
int dcmp(double x, double y) { return dcmp(x - y); }

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point& operator=(const Point& p) {
        x = p.x, y = p.y;
        return *this;
    }
};
typedef Point Vector;

Vector operator+(const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator-(const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
Vector operator*(const Vector& A, double p)
{ return Vector(A.x * p, A.y * p); }
bool operator==(const Point& a, const Point& b)
{ return a.x == b.x && a.y == b.y; }
bool operator<(const Point& p1, const Point& p2) {
    if (p1.x != p2.x) return p1.x < p2.x;
    return p1.y < p2.y;
}
double Dot(const Vector& A, const Vector& B) { return A.x * B.x + A.y * B.y; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
double Angle(const Vector& A, const Vector& B)
{ return acos(Dot(A, B) / Length(A) / Length(B)); }
double Cross(const Vector& A, const Vector& B)
{ return A.x * B.y - A.y * B.x; }

```

```

Vector Rotate(Vector A, double rad)
{ return Vector(A.x*cos(rad) - A.y*sin(rad), A.x*sin(rad) + A.y*cos(rad)); }

Vector Normal(Vector A) {
    double L = Length(A);
    return Vector(-A.y / L, A.x / L);
}

bool SegmentProperIntersection(const Point& a1, const Point& a2,
const Point& b1, const Point& b2) {
    double c1 = Cross(a2 - a1, b1 - a1), c2 = Cross(a2 - a1, b2 - a1),
        c3 = Cross(b2 - b1, a1 - b1), c4 = Cross(b2 - b1, a2 - b1);
    return dcmp(c1) * dcmp(c2) < 0 && dcmp(c3) * dcmp(c4) < 0;
}

Point GetLineIntersection(Point P, Vector v, Point Q, Vector w) {
    Vector u = P - Q;
    double t = Cross(w, u) / Cross(v, w);
    return P + v * t;
}

bool OnSegment(const Point& p, const Point& a1, const Point& a2) {
    return dcmp(Cross(a1 - p, a2 - p)) == 0 && dcmp(Dot(a1 - p, a2 - p)) < 0;
}

istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
ostream& operator<<(ostream& os, const Point& p)
{ return os << p.x << " " << p.y; }

int main() {
    int N;
    for (int t = 1; cin >> N && N; t++) {
        Point p;
        set<Point> all_points;
        vector<Point> ps;
        _for(i, 0, N) cin >> p, ps.push_back(p), all_points.insert(p);
        int E = --N;
        _for(i, 0, N) _for(j, i + 1, N)
            if (SegmentProperIntersection(ps[i], ps[i + 1], ps[j], ps[j + 1]))
                all_points.insert(
                    GetLineIntersection(ps[i], ps[i + 1] - ps[i], ps[j], ps[j + 1] - ps[j]));

        for(set<Point>::iterator si = all_points.begin();
            si != all_points.end(); si++)
            _for(i, 0, N)
                if(OnSegment(*si, ps[i], ps[i + 1])) E++;
        int F = E + 2 - all_points.size(); // V+F-E=2, 点, 面, 边
        printf("Case %d: There are %d pieces.\n", t, F);
    }
    return 0;
}

```